Fast Fourier Transform

Tanvir Ahmed

Math Survey 2025-26

§1 Fourier Series and Fourier Transforms

The idea of a Fourier Series is to be able to write a function (usually periodic) as an infinite sum of sine and cosine terms, similar to how a Power Series is an infinite sum of polynomial terms. Because sin and cos can be expressed as complex exponentials, particularly:

$$\cos x = \frac{e^{ix} + e^{-ix}}{2}$$
$$\sin x = \frac{e^{ix} - e^{-ix}}{2i},$$

for our purposes, we will treat a Fourier Series of a function f as an infinite sum of e^{inx} terms for integers n. In other words,

$$f(x) = \sum_{n=-\infty}^{\infty} a_n e^{inx}$$

would be a Fourier series for $f(x)$. The following theorem gives a way to actually calculate the Fourier Coefficients a_n.

Theorem 1.1

For a real number $L > 0$ and a continuous function $f : [-L, L] \to \mathbb{C}$, if there is a Fourier Series for f, i.e.

$$f(x) = \sum_{n=-\infty}^{\infty} a_n e^{in\pi x/L}$$

for some choice of $a_n \in \mathbb{C}$, then for $n \in \mathbb{Z}$,

$$a_n = \frac{1}{2L} \int_{-L}^{L} f(t) e^{-in\pi t/L}\, dt.$$

Proof. Suppose we can write $f(x)$ in the desired form and converges over all of $[-L, L]$. Notice that for $m \neq n$, we have

$$\int_{-L}^{L} e^{i(m-n)t/L}\, dt = \frac{1}{i(m-n)/L} e^{i(m-n)t/L}\Big|_{-L}^{L} = 0.$$

Now consider

$$f(t) = \sum_{n=-\infty}^{\infty} a_n e^{in\pi x/L}.$$

To extract the coefficient a_n, we can multiply both sides by $e^{-int/L}$ and integrate from $-L$ to L. Every term with index not equal to n vanishes due to the above integral and we are left with

$$\int_{-L}^{L} f(t)e^{-in\pi t/L}\,dt = \int_{-L}^{L} a_n\,dt = 2La_n.$$

Dividing by $2L$ gives the desired conclusion. □

Based on this technique we used to extract coefficients for a Fourier Series, we define the Fourier Transform as follows:

Definition 1.2 — Given a complex-valued function $f(x)$ on the real line, the complex-valued function $\hat{f}(\xi)$, the Fourier transform of $f(x)$, is defined as

$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(x)e^{-2i\pi\xi x}\,dx.$$

Roughly speaking, the Fourier Transform can detect when a certain frequency is present in a function. For example, if $\hat{f}(2)$ is very big (in magnitude) compared to the values of $\hat{f}$ around 2, then there is probably a large similarity of f to $e^{4\pi i}$, which would indicate a frequency of 2. Sometimes we may also denote $\hat{f}$ by $\mathcal{F}(f)$.

§2 Discrete Fourier Transform

Now we define the Discrete Fourier Transform (DFT), which is a discrete analogue of the continuous Fourier Transform, given in Definition 1.2. In the continuous case, we integrate a function f times an exponential factor over the entire real line. For the discrete case, we turn the integral into a sum, and the values of the exponential term we use end up being the nth roots of unity.

Definition 2.1 — Suppose f is a complex-valued function on the set $\{0, 1, \ldots, n-1\}$ for some positive integer n. Then, for integers k, define the *Discrete Fourier Transform* of f as

$$\hat{f}(k) = \sum_{j=0}^{n-1} f(j)e^{-2\pi ijk/n}.$$

Notice in the definition that f is defined on $\{0, 1, \ldots, n-1\}$. A better way to phrase is that f is defined on all integers but its value only depends on its input mod n. In other words, f is defined on the integers mod n, which we denote as $\mathbb{Z}/n\mathbb{Z}$. Note that the same is true of $\hat{f}$ since plugging since the exponential term in $\hat{f}$ cycles as k cycles mod n. We can write the definition for $\hat{f}$ more concisely by using roots of unity. In particular, let $\omega = e^{-2\pi i/n}$, which is a primitive nth root of unity. Then, we have

$$\hat{f}(k) = \sum_{j=0}^{n-1} f(j)\omega^{jk}.$$

When we are using the Discrete Fourier Transform, we are often considering all the values of f at once and trying to compute the values of $\hat{f}$. Thus, it is useful to think of the Discrete Fourier Transform as a matrix whose input is a vector of the values of f and whose output is the a vector of the values of $\hat{f}$. The above formula then tells us that the coefficient in the jth row and kth column

of the matrix is exactly ω^{jk}. In other words, we have the following matrix equation:

$$\begin{pmatrix} \hat{f}(0) \\ \hat{f}(1) \\ \hat{f}(2) \\ \vdots \\ \hat{f}(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{n-2} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{n-3} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{n-2} & \omega^{n-3} & \dots & \omega \end{pmatrix} \begin{pmatrix} f(0) \\ f(1) \\ f(2) \\ \vdots \\ f(n-1) \end{pmatrix}$$

Notice that we can reduce the powers of ω in the matrix mod n since ω is an nth root of unity.

§2.1 Inverse DFT

Given the DFT of a function $f : \mathbb{Z}/n\mathbb{Z} \to \mathbb{C}$, we can apply a simple formula to obtain the original values for f that almost looks like applying the DFT again. In fact, we will prove the following:

Theorem 2.2 (Inverse DFT)

For a function $f : \mathbb{Z}/n\mathbb{Z} \to \mathbb{C}$ and $k \in \mathbb{Z}/n\mathbb{Z}$, we have

$$f(k) = \frac{1}{n} \sum_{j=0}^{n-1} \hat{f}(j)\omega^{-jk}.$$

In other words, this means that $f = \frac{1}{n}\hat{\hat{f}}$, so taking an inverse FFT is essentially equivalent to taking a standard FFT.

Proof. We will consider all k at once and think of the above as a matrix equation. The left hand side is just a vector containing all the values of f. The right hand side is

$$\frac{1}{n} \begin{pmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3} & \dots & \omega \\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6} & \dots & \omega^2 \\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9} & \dots & \omega^3 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega & \omega^2 & \omega^3 & \dots & \omega^{-1} \end{pmatrix} \begin{pmatrix} \hat{f}(0) \\ \hat{f}(1) \\ \hat{f}(2) \\ \vdots \\ \hat{f}(n-1) \end{pmatrix}$$

$$= \frac{1}{n} \begin{pmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3} & \dots & \omega \\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6} & \dots & \omega^2 \\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9} & \dots & \omega^3 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega & \omega^2 & \omega^3 & \dots & \omega^{-1} \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{n-2} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{n-3} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{n-2} & \omega^{n-3} & \dots & \omega \end{pmatrix} \begin{pmatrix} f(0) \\ f(1) \\ f(2) \\ \vdots \\ f(n-1) \end{pmatrix}$$

We want to show that $\frac{1}{n}$ times the two matrices in the middle turn out to be the identity matrix I. Recall that if we have vectors A and B, the entry in the ith row and jth column of AB is the dot product of the ith row of A and the jth column of B. We have to show that this dot product is 0 for $i \neq j$ and n for $i = j$ (for this particular case of matrices). Note we will be zero-indexing our rows and columns for the proceeding calculations, since everything else is zero-indexed.

For $i \neq j$, the ith row of the left matrix is $(\omega^{i\ell})_\ell$, and the jth column of the right matrix is $(\omega^{-j\ell})_\ell$. The dot product of these is

$$\sum_{\ell=0}^{n-1} \omega^{i\ell}\omega^{-j\ell} = \sum_{\ell=0}^{n-1} \omega^{(i-j)\ell}.$$

Since $i - j \neq 0$, ω^{i-j} is a root of unity other than 1, so when summing all its powers, we get 0. Now we have the other case when $i = j$. Then, the above dot product we computed is just

$$\sum_{\ell=0}^{n-1} 1 = n$$

as desired. □

§3 Fast Fourier Transform

Computing the DFT for an input of size n naively requires $O(n^2)$ operations since each entry of the DFT matrix needs to get multiplied by some value in the input once. However, this matrix has a lot of symmetry which we aren't using. The Fast Fourier Transform is an algorithm that leverages this symmetry to compute the DFT in a more efficient time complexity of $O(n \log n)$.

§3.1 The Cooley-Tukey Algorithm

The key idea of FFT is to reduce the computation of an N-point FFT into two computations of $\frac{N}{2}$-point FFT's. For now, we will assume that N is even for convenience. Then, if we want to take the FFT of a function f which is defined on $\mathbb{Z}/N\mathbb{Z}$, notice that we can split up the even and odd terms in the definition of $\hat{f}$ as follows:

$$\begin{aligned}\hat{f}(k) &= \sum_{j=0}^{N-1} f(j)\omega_N^{jk} \\ &= \sum_{j=0}^{\frac{N}{2}-1} f(2j)\omega_N^{2jk} + \sum_{j=0}^{\frac{N}{2}-1} f(2j+1)\omega_N^{(2j+1)k}.\end{aligned}$$

Then, notice that $\omega_N^2 = \omega_{N/2}$, so this is equal to

$$\sum_{j=0}^{\frac{N}{2}-1} f(2j)\omega_{N/2}^{jk} + \omega_N^k \sum_{j=0}^{\frac{N}{2}-1} f(2j+1)\omega_{N/2}^{jk}.$$

These two sums are essentially $\frac{N}{2}$-point FFT's–one of them for the vector $[f(2j)]$ and the other for the vector $[f(2j+1)]$. This algorithm works by recursively computing these smaller FFT's using the same subdividing strategy. In the simplest case, we have that N is a power of 2. We would like to analyze the efficiency of this algorithm in this case. We will assume that all additions and multiplications take the same amount of time.

Suppose that $N = 2^n$ and let a_n be the number of operations necessary for a 2^n-point FFT. To compute this FFT, we must compute two 2^{n-1}-point FFT's as well as the values of $1, \omega_N, \ldots, \omega_N^{N-1}$. Then, to compute each value $\hat{f}(k)$, we just have to add a pre-computed value of the FFT of $[f(2j)]$,

a pre-computed value of ω_N^k, and a pre-computed value of the FFT of $[f(2j+1)]$. Overall, the number of operations used is $2a_{n-1} + cN = 2a_{n-1} + c \cdot 2^n$ where c is some constant.

We thus have the recurrence $a_n = 2a_{n-1} + c \cdot 2^n$. We can actually solve this recurrence. Let $A(x)$ be the generating function for the sequence a_n. In other words,

$$A(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Taking out the first term, we can apply the recurrence to get

$$\begin{aligned} A(x) &= a_0 + \sum_{n=1}^{\infty} (2a_{n-1}x^n + c \cdot 2^n x^n) \\ &= a_0 + x\sum_{n=0}^{\infty} 2a_n x^n + c\sum_{n=1}^{\infty} (2x)^n \\ &= a_0 + 2xA(x) + \frac{2cx}{1-2x}. \end{aligned}$$

We can solve for $A(x)$ to get

$$A(x) = \frac{a_0}{1-2x} + \frac{2cx}{(1-2x)^2}.$$

From this formula, we can extract the value of a_n by expanding out the geometric series. The contribution of the term $\frac{a_0}{1-2x}$ to the coefficient of x^n in $A(x)$ is exactly $a_0 \cdot 2^n$, and it is not too hard to check that the contribution of the second term is $c(n+1) \cdot 2^n$. Therefore, we have $a_n = c(n+1) \cdot 2^n + a_0 \cdot 2^n$. In terms of N, which we are measuring the time complexity based off, this is equal to $cN(\log_2 N + 1) + a_0 N$. Thus, the time complexity of the Cooley-Tukey Algorithm is on the order of $O(N \log N)$. This means that multiplying by the DFT matrix can be done much more efficiently than the naive $O(n^2)$ solution. We will more thoroughly discuss the implications of this algorithm and its applications in the modern-day in later sections.

§4 Non-powers of 2

One limitation of the Cooley-Tukey Algorithm is that it only works for powers of 2. The algorithm can also be used to decompose a N-point FFT into N_1 N_2-point FFT's where $N = N_1 N_2$. However, for this decomposition to be effective, it is best for N to have only small prime factors. This way, one can decompose the N-point FFT into small pieces eventually. However, we would like to come up with an FFT algorithm that works for any N, especially large primes N. We will accomplish this by using Bluestein's Algorithm and the Convolution Theorem.

§4.1 Convolution Theorem

One tool we will need is the Convolution Theorem. This is an identity we can use for both continuous and discrete Fourier Transforms, so we introduce both cases. Suppose we have two sufficiently nice functions $u, v : \mathbb{R} \to \mathbb{C}$ and define their convolution r as the function

$$r(x) = \int_{-\infty}^{\infty} u(t)v(x-t)\, dt.$$

Notice that if we switch the order and u and v, r stays the same, which we can prove by making a substitution in the integral. We will also use the notation $r = u * v$.

Theorem 4.1 (Convolution Theorem)
Given the definitions earlier, we have $\hat{r}(\xi) = \hat{u}(\xi)\hat{v}(\xi)$ for all $\xi \in \mathbb{R}$. In other words, $\widehat{u * v} = \hat{u} \cdot \hat{v}$.

Proof. We can just carry out the computation:

$$\begin{aligned}
\hat{r}(\xi) &= \int_{-\infty}^{\infty} r(t)e^{-2\pi i\xi t}\, dt \\
&= \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} u(q)v(t-q)\, dq\, e^{-2\pi i\xi t}\, dt \\
&= \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} u(q)v(t-q)e^{-2\pi i\xi q}e^{-2\pi i\xi(t-q)}\, dt\, dq \\
&= \int_{-\infty}^{\infty} u(q)e^{-2\pi i\xi q}\left(\int_{-\infty}^{\infty} v(t-q)e^{-\pi i\xi(t-q)}\, dt\right) dq \\
&= \int_{-\infty}^{\infty} u(q)e^{-2\pi i\xi q}\hat{v}(\xi)\, dq \\
&= \hat{u}(\xi)\hat{v}(\xi).
\end{aligned}$$

□

This can be generalized to convolutions of discrete functions using the Discrete Fourier Transform. In particular, let f, g be functions defined on $\mathbb{Z}/n\mathbb{Z}$, and define their convolution h as

$$h(k) = \sum_{j=0}^{n-1} f(j)g(k-j).$$

Notice that the argument of g will sometimes be negative, but that is fine since f and g are just defined mod n. The discrete version of the Convolution Theorem states that $H(k) = F(k)G(k)$ where F, G, and H are the DFT's of f, g, and h respectively. This is not too hard to prove, and is quite similar to the continuous case:

$$\begin{aligned}
H(k) &= \sum_{j=0}^{n-1} h(j)\omega^{jk} \\
&= \sum_{j=0}^{n-1}\sum_{\ell=0}^{n-1} f(\ell)g(j-\ell)\omega^{jk} \\
&= \sum_{\ell=0}^{n-1} f(\ell)\omega^{\ell k}\sum_{j=0}^{n-1} g(j-\ell)\omega^{(j-\ell)k} \\
&= \sum_{\ell=0}^{n-1} f(\ell)\omega^{\ell k}G(k) \\
&= F(k)G(k).
\end{aligned}$$

A useful way to write the convolution theorem for computational purposes is:

Theorem 4.2

Let $f, g : \mathbb{Z}/n\mathbb{Z} \to \mathbb{C}$ be functions defined mod n. Then, $f * g = \mathcal{F}^{-1}(\mathcal{F}(f)\mathcal{F}(g))$.

Therefore, rather than computing convolutions directly using the definition, we can just perform two FFT's and one inverse FFT.

§4.2 Bluestein's Algorithm

One limitation to the Cooley-Tukey Theorem is that it only works for composite values of N, and in the case above, we only considered it when N is a power of 2. The worst case is when N is a large prime number, and we can't factor it and simplify the computation of the FFT using the Cooley-Tukey Theorem. Bluestein's Algorithm gets around this by reducing general FFT's to FFT's of power-of-2 order. The main idea is to write the DFT as a convolution of other sequences and then proceed to use Theorem 4.2 and the Cooley-Tukey Algorithm to finish.

In particular, suppose we are given a function $f : \mathbb{Z}/N\mathbb{Z} \to \mathbb{C}$ and let ω be an Nth root of unity. We are trying to compute

$$\hat{f}(k) = \sum_{j=0}^{n-1} f(j)\omega^{jk}.$$

Now observe that

$$jk = \frac{-(k-j)^2}{2} + \frac{j^2}{2} + \frac{k^2}{2}.$$

This allows us to write the desired DFT as

$$\begin{aligned}\hat{f}(k) &= \sum_{j=0}^{n-1} f(j)\omega^{jk} \\ &= \omega^{\frac{k^2}{2}} \sum_{j=0}^{n-1} \left(f(j)\omega^{\frac{j^2}{2}} \right) \omega^{-\frac{(k-j)^2}{2}}.\end{aligned}$$

Notice that we have now written $\hat{f}(k)$ as a convolution of two sequences, which we will define as

$$a_m = f(m)\omega^{\frac{m^2}{2}}$$

and

$$b_m = \omega^{-\frac{m^2}{2}}.$$

In other words, we have

$$\hat{f}(k) = \omega^{\frac{k^2}{2}} \sum_{j=0}^{n-1} a_j b_{k-j} = \omega^{\frac{k^2}{2}} (a * b)_j.$$

where $a * b$ is the convolution of a and b. Thus, if we can efficiently compute the sequence $a * b$, we can efficiently compute the Fourier transform of f. We can use Theorem 4.2 to do this, turning the operation of convolution into a few FFT's. However, the problem is that these FFT's don't operate on a vector with a power-of-2 number of elements. We can solve this problem by zero-padding. As an example, suppose we were trying to find the convolution of two sequences a_k and b_k which both only have 5 terms.

a_1	a_2	a_3	a_4	a_5
b_1	b_2	b_3	b_4	b_5

In the diagram above, we have the sequences a_k on top of b_k and the terms which are color coded will be multiplied together in the calculation of $(a * b)_1$. Note that for example $a_1 b_5$ is included because we add the indices mod 5. We can pad both sequences with 0's in the following way to have the same calculation:

a_1	a_2	a_3	a_4	a_5	0	0	0
0	0	0	b_1	b_2	b_3	b_4	b_5

Now we are calculating $(a * b)_1$ where we consider a and b to be the 8-term sequences as above. Note that now $a_1 b_5$ is still included in the calculation since a_1 is the 1st term of a and b_5 is now the 8th term of b, and $1 + 8 \equiv 1 \pmod 9$. Because of the 0's we added, we always get the same pairs of terms as we did in the 5-term case, even if we were to compute a different term of $a * b$. Thus, if we want to compute the convolution of two sequences of arbitrary length, it suffices to do this zero-padding to a power-of-two length, and then apply Theorem 4.2 with the Cooley-Tukey Algorithm to efficiently perform FFT's. Going back, this allows us to perform arbitrary-length DFT's in $O(n \log n)$ time complexity, albeit with a considerably larger constant factor than for powers of 2.

§5 Implementation

In this section, we will show an implementation of the FFT and related algorithms we have discussed. Our input to the FFT will be an array of numbers; for the Cooley-Tukey Algorithm, we must assume that the length of the array is a power of 2. In the Cooley-Tukey Algorithm, we split up these values into the even and odd indices, perform the FFT on both parts, and then take a linear combination of the results for our final answer. This is essentially the structure that our code will take.

```
import numpy as np
def FFT(x):
    N = len(x)

    if N == 1:
        return x
    else:
        even = FFT(x[::2])
        odd = FFT(x[1::2])
        factor = np.exp(-2j*np.pi*np.arange(N)/ N)

        X = np.concatenate(
            [even+factor[:int(N/2)]*odd,
             even+factor[int(N/2):]*odd])
        return X

print(FFT([1, 2, 1, 1]))
```

Clearly, this can be expanded to an implementation of Bluestein's algorithm for arbitrary length FFT's through the use of convolutions we explained in the previous section.

In python, the FFT can be used easily using the `numpy` package. In particular, `numpy` allows for easy use of many math-related commands, using special `numpy` arrays. To use the FFT, you must use the `numpy.fft.fft` command on an array constructed using the `np.array` function. Here is an example:

```python
import numpy as np
x = np.array([1, -1, 2, 6, 4])
print(np.fft.fft(x))

# output: [12.+0.j, -4.54508497+7.10642359j,
# 1.04508497-0.8652998j, 1.04508497+0.8652998j,
# -4.54508497-7.10642359j]
```

`numpy.fft` also includes other useful commands such as `numpy.fft.ifft` (Inverse FFT), `numpy.fft.fft2` (2-dimensional FFT), and `numpy.fft.fftn` (n-dimensional FFT). Furthermore, it is important to remember that in python, such as in the output provided in the above example code, the imaginary unit i is displayed using the letter `j`. Regarding the actual output, we can see that the first element of FFT output is the sum of all the elements in `x` since it is really the sum of each element of the array times $\omega^0 = 1$, where $\omega = e^{-\frac{2\pi i}{5}}$.

§6 Multiplication

§6.1 Polynomial Multiplication

Suppose we have a polynomial $A(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_0$. We will denote its coefficient vector, containing the a_i's as just A. If $\omega = e^{-\frac{2\pi i}{n}}$ as a usual, we have $\hat{A}(0) = a_n\omega^0 + a_{n-1}\omega^0 + \cdots + a_0 = A(1) = A(\omega^0)$. In fact, we have that $\hat{A}(k) = A(\omega^k)$ in general. Now, suppose we have another polynomial $B(x) = b_m x^m + b_{m-1}x^{m-1} + \cdots + b_0$. Consider taking the FFT of the product $A(x)B(x)$. The result will consist of values $A(\omega^k)B(\omega^k)$ for a root of unity ω. However, we can find all the values of $A(\omega^k)$ and $B(\omega^k)$ fast by FFT'ing them first. From there, we can take the IFFT of the resulting vector.

One thing we must do first is to zero-pad the coefficients for A and B. If A has degree n and B has degree m, then we will zero-pad both vectors so they have a total length L where L is a power of 2 greater than or equal to $m + n$. We want L to be a power of 2 since FFT's are fastest when the length of the input is a power of 2 (using Cooley-Tukey). Furthermore, we want $L \geq m + n$ since the length of the coefficient vector AB will be $m + n$. Then, $\mathcal{F}(A)$ is a vector consisting of the values of $A(\omega^k)$ where $\omega = e^{-\frac{2\pi i}{L}}$, and similarly for $\mathcal{F}(B)$. Then, notice that $\mathcal{F}(AB)$ consists of the values of $A(\omega^k)B(\omega^k)$, so $\mathcal{F}(AB) = \mathcal{F}(A) \cdot \mathcal{F}(B)$, where the dot represents the pairwise products of two vectors. Finally, taking the IFFT, we have $AB = \mathcal{F}^{-1}(\mathcal{F}(A) \cdot \mathcal{F}(B))$. All these operations can be computed in $O(n \log n)$ time, so we have an algorithm for polynomial multiplication that is $O(n \log n)$. Note that the naive algorithm with distributing all the coefficients is $O(n^2)$, so this is a significant improvement.

§6.2 Schönhage-Strassen Algorithm

The Fast Fourier Transform can be used to construct an efficient algorithm for multiplication of large numbers, known as the Schönhage-Strassen Algorithm. First of all, note that the ordinary algorithm for multiplying two n-digit numbers (using column multiplication) has time complexity $O(n^2)$ because for each digit in the first number, we must multiply it by each digit in the second number. The first algorithm with a faster time complexity for this problem was Karatsuba's Algorithm, which divides the multiplication of two n-digit numbers into 3 multiplications of two $n/2$-digit numbers (rather than 4). This achieves a time complexity of $O(n^{\log_2 3}) \approx O(n^{1.58})$ multiplications of 1-digit numbers.

The Toom-Cook Algorithm uses a similar approach and for example, can reach a time complexity of $O(n^{\log_3 5}) \approx O(n^{1.46})$.

On the other hand, the Schönhage-Strassen Algorithm uses the FFT to achieve a time complexity of $O(n \log n \log\log n)$. In this algorithm, we calculate the product of two numbers a and b mod a number of the form $2^N + 1$.

In particular, we want to choose some large N where $N = 2^k \cdot m$. Then, we split a and b into $D = 2^k$ blocks of m bits each. These D blocks for a and b can be interpreted as vectors of length D, and multiplying a and b essentially comprises of taking a convolution of these two vectors, so we would like to use the Convolution Theorem to do so. Computing the FFT of these two vectors will require a Dth primitive root. It is not too hard to see that $g = 2^{2m}$ is a Dth primitive root of unity mod $2^N + 1$, so we will use this for the FFT used in the Convolution Theorem, which will give us the final product.

First of all, for this setup to work, we must have $ab < 2^N + 1$. From there, the FFT's used for the Convolution Theorem are of length 2^k, so the time complexity should be $O(k \cdot 2^k)$. However, we must remember that the FFT requires us to multiply many m-digit numbers, so the time complexity of multiplication can't be treated as constant (after all, we are describing a multiplication algorithm). While we won't go over the details here, by finding the optimal value of k and m, this algorithm works with time complexity $O(n \log n \log\log n)$ where n is the number of digits.

The Schönhage-Strassen only starts to get faster than other previously mentioned algorithms like Tom-Cook for significantly large numbers, but they are not unfathomably large (we start seeing major improvements with 100000+ digits). In fact, this algorithm has been used in the Great Internet Mersenne Prime Search and calculating approximations of π. There is another theoretical algorithm for multiplication with a time complexity of $O(n \log n)$ known as the Harvey-Hoeven Algorithm

§7 Signal Processing

The Discrete Fourier Transform, as the name suggests, is just a discrete version of the continuous Fourier Transform. This makes it perfect for working with digital data. Since running a Fourier Transform on a wave can give you information about its component frequencies, the DFT can do this for digital data. One particular example of this is essential for digital communications.

§7.1 Orthogonal Frequency-Division Multiplexing

Orthogonal Frequency-Division Multiplexing (OFDM) is a crucial technology that allows for wireless communications as we know it. When sending a wireless signal, one must transmit a wave or collection of waves through the air, known as carrier waves. Suppose we are trying to send a series of bits in this way. One way we can do this is by fixing some frequency for the wave, and for every 1 in the signal, the wave has a certain phase, and for every 0, the wave has a certain different phase. This method is not very favorable because every bit of information has to be sent separately in time, and there is no way to send them simultaneously.

One solution to this is to send data that is encoded by the frequency of carrier waves, leading us to Frequency-Divison Multiplexing. Suppose we wanted to send a sine wave given by $\sin(2x)$. This is a linear combination of e^{2ix} and e^{-2ix}, so the Fourier Transform will have peaks at $2i$ and $-2i$. In other words, these are the frequencies inherent to the sine function.

However, in reality, we would only be sending a sine wave for a limited amount of time, and when applying the Fourier Transform on this signal, the most apparent frequencies are still $\pm 2i$, but there are still nonzero contributions from other frequencies.

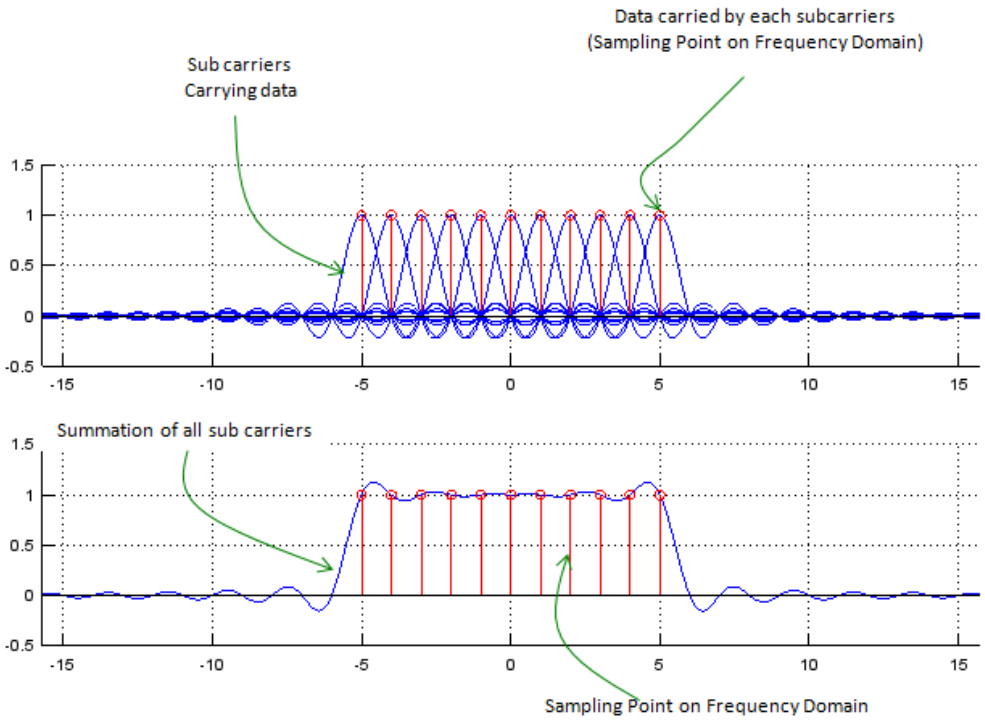


Figure 1: The range of frequencies used by different OFDM subcarriers

Now suppose we want to transmit data using different frequencies. For the sender, this would just consist of adding waves of the particular frequencies. The receiver of the signal can then use the Fourier Transform to extract the frequencies themselves. In practice, this will be done using the FFT since all digital waves are received as discrete signals. However, as mentioned earlier, the Fourier transforms of the transmitted waves are not just singular peaks, so the presence of one wave could possibly interfere with the presence of another if they share frequencies in the Fourier Transform.

As a result, the distinct frequencies used in this type of signal transmission have to be spaced out further apart so that they don't interfere with one another. This increases the total bandwidth, the overall range of frequencies that a device can transmit. Bandwidth is costly, so we would like to have a way to decrease it.

Figure 7.1 shows how the frequencies used in OFDM are arranged. The top graph shows different frequencies that can be transmitted (each by a different subcarrier), and they overlap. However, when one frequency is at its peak, all others are 0, so in the transmitted signal, each frequency can still be detected independently from the others. Furthermore, keep in mind that when the transmitted frequencies are selected, they have to be converted from the frequency domain to the time domain, which requires an inverse FFT. At the receiving end, the signal just needs to go through a regular FFT.

Compared to other types of digital signals, OFDM is signals are more resistant from changes due to the environment (such as bouncing off objects) since such environmental factors typically don't change the frequencies of waves. This has caused OFDM to be crucial for technologies like Wi-Fi and 5G, which we use every day.

§8 Data Compression

§8.1 Multi-Dimensional FFT

So far, we have been considering FFT's of vectors of complex numbers. First, we will generalize the DFT to multiple dimensions. For example, let us consider a 2-dimensional DFT. In this case, we consider a 2-variable function $f(x, y) : \mathbb{Z}/n\mathbb{Z} \times \mathbb{Z}/m\mathbb{Z} \to \mathbb{C}$, and we have

$$\hat{f}(k_1, k_2) = \sum_{i \in \mathbb{Z}/n\mathbb{Z}} \sum_{j \in \mathbb{Z}/m\mathbb{Z}} f(i, j)\omega_n^{ik_1}\omega_m^{jk_2}$$

where $\omega_n = e^{-2\pi i/n}$ and $\omega_m = e^{-2\pi i/m}$ as usual. The general multi-dimensional case is analogous to this, but with a lot more summations.

Definition 8.1 — Let $f(x_1, \ldots, x_\ell) : \mathbb{Z}/n_1\mathbb{Z} \times \cdots \times \mathbb{Z}/n_\ell\mathbb{Z} \to \mathbb{C}$ be a function. We define its Disrete Fourier Transform as the function

$$\hat{f}(k_1, \ldots, m_\ell) = \sum_{i_1 \in \mathbb{Z}/n_1\mathbb{Z}} \cdots \sum_{i_\ell \in \mathbb{Z}/n_\ell\mathbb{Z}} f(i_1, \ldots, i_\ell)\omega_{n_1}^{i_1k_1} \ldots \omega_{n_\ell}^{i_\ell k_\ell}.$$

It turns out that performing a 2-dimensional DFT is the same as performing a DFT of the columns first and then on the rows, as stated in the following theorem. An analogous statement is true for n-dimensional DFT's

Theorem 8.2

Consider a function $f(x, y) : \mathbb{Z}/n\mathbb{Z} \times \mathbb{Z}/m\mathbb{Z} \to \mathbb{C}$. Then, the 2D array of values given by $[\hat{f}(k_1, k_2)]$ is the same as the result of applying a DFT to every row of $[f(x, y)]$ and then to every column, or vice versa.

§8.2 Image Compression

The JPEG Image Compression algorithm can typically compress the size of an image by 10 times with virtually no loss to the image quality. The simple idea of how this compression is by using a 2-dimensional FFT on the image, with a couple of nuances.

First of all, the JPEG algorithm actually uses the DCT, or the Discrete Cosine Transform, which is similar to the DFT as both convert the information of the pixels into the frequency domain. From here, the JPEG algorithm essentially throws away high frequency information in the data, which don't really effect our eyes' perception of the image. For example, high frequency data might include subtle changes in the color of the background whereas lower frequencies might account for the boundaries of different objects. This information is discarded by being stored less precisely the lower frequency information.

JPEG compression divides an image into 8×8 squares of pixels and for each square, performs the DCT on the Y, C_b, and C_r components. Roughly speaking, the Y component is the greyscale version of the image while the C_b and C_r components are the blue and red chrominance components, as shown in Figure 8.2.

Figure 8.2 shows the 64 fundamental frequencies that make up 8×8 images. Each cell of the grid represents a different pair of frequencies, one for each dimension of the grid. The frequencies on the top and left sides are more significant as they carry less noise, so the JPEG algorithm prioritizes these more.

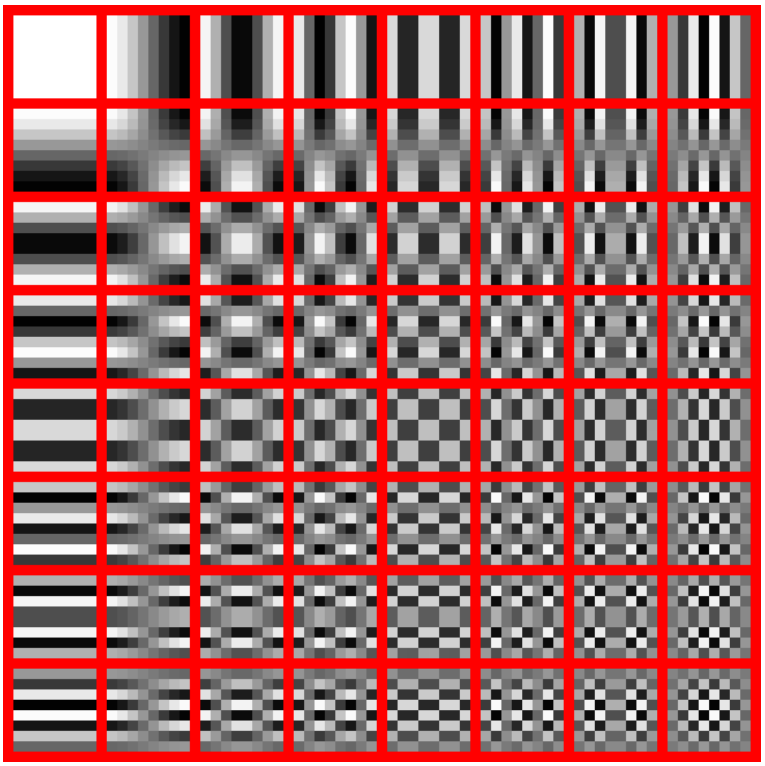


Figure 2: The 64 fundamental frequencies on an 8×8 grid of pixels

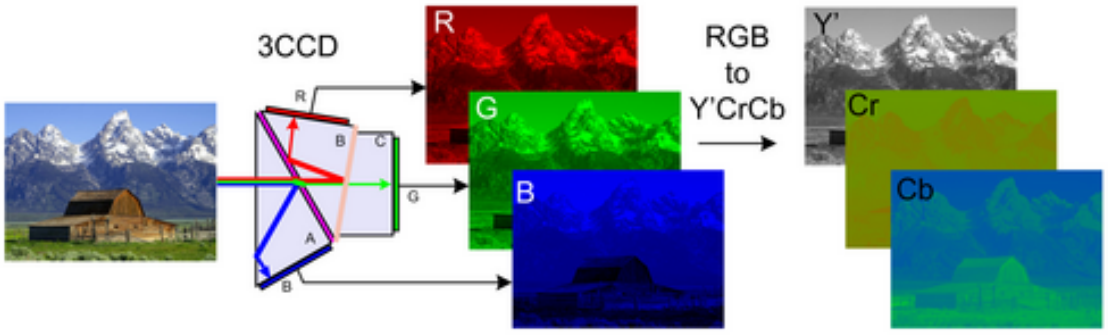


Figure 3: An image split into its Y, C_b, and C_r components

§9 Conclusion

Overall, the FFT takes advantage of the symmetry present in roots of unity and the calculation of a DFT, allowing for a very large speed improvement from $O(n^2)$ to $O(n \log n)$. Because the DFT is such an essential operation in signal processing, so many modern technologies can only function because of the speedup provided by the FFT. If we were stuck with a $O(n^2)$ complexity, we wouldn't have the luxury of fast wireless communications or near-flawless data compression. As Gilbert Strang, a mathematics professor at MIT said, the FFT "is the most important numerical algorithm of our lifetime."